CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT
ON

Air Quality Data Analysis Using Python

Python Programming / Data Analytics

PROJECT REPORT

SUBMITED BY:

Name : MAYANK SHARMA **UID : 25BCD10113**

Branch : BCA DATA SCIENCE

Group: B **Section: 25BCD**

Semester : 3RD (2 YEAR)

SUBMITED TO:

Name :MRS. PREETI **D.O.SUBMITION : 8TH OCTOBER 2026**

Desigination: PROFESSSOR **Sign:**_____

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Project Report: Air Quality Data Analysis Using Python

1. Title

Air quality is a major environmental and public-health concern because polluted air can affect respiratory health, cardiovascular health, ecosystems, visibility, and quality of life. This case study demonstrates how Python can be used to organise, clean, analyse, and visualise air-quality observations. The project focuses on a reproducible workflow rather than claiming a specific city-level result without a verified dataset.

The analysis can use measurements such as PM2.5, PM10, nitrogen dioxide (NO_2), sulphur dioxide (SO_2), carbon monoxide (CO), ozone (O_3), temperature, humidity, wind speed, date, and monitoring station. The exact variables depend on the dataset available to the student.

Case-study scope

The project covers dataset loading, data-quality checks, missing-value handling, descriptive statistics, time-based trends, pollutant comparisons, visualisation, and optional export of cleaned results. It is suitable for an introductory Python/data analytics project and can be extended with forecasting or geographic analysis when adequate data are available.

2. Aim / Introduction of the Project

The aim of this project is to apply Python programming and data-analysis techniques to understand patterns in air-quality data. Air pollution is not constant: concentrations may vary by hour, day, season, weather conditions, traffic activity, industrial activity, and location. A structured analysis can help transform raw sensor readings into information that is easier to interpret.

Raw environmental datasets often contain inconsistent date formats, missing observations, duplicate rows, incorrect data types, and unusually high or low values. Analysing the data before checking these issues can produce misleading summaries. Therefore, this case study follows a staged process: inspect the dataset, standardise column names, convert dates and numeric measurements, assess data quality, document cleaning decisions, and only then calculate statistics and create charts.

Python is appropriate for this work because it has libraries for data manipulation, numerical calculation, visualisation, and scientific computing. Pandas provides DataFrame operations; NumPy supports numerical operations; Matplotlib and Seaborn help communicate patterns through charts. The same workflow can be rerun when new observations are added, improving consistency and reproducibility.

This report does not assume that any one pollutant is always the most harmful or that a chart alone proves the cause of pollution. Results must be interpreted in the context of measurement units, sensor quality, station location, time period, and the relevant air-quality standards. If the dataset is a sample or demonstration dataset, its findings should not be presented as official environmental measurements.

Problem statement

How can Python be used to process air-quality observations and reveal useful patterns in pollutant concentrations while reducing errors caused by missing, duplicated, or incorrectly formatted data?

Proposed solution

Develop a repeatable analysis pipeline that loads a CSV file, validates expected fields, cleans common data issues, calculates summary statistics, identifies high readings for further investigation, visualises trends, and exports the processed dataset and figures.

3. Objective

- Import air-quality observations from a CSV file into a Pandas DataFrame.
- Understand the dataset structure, columns, units, date range, and number of observations.
- Identify missing values, duplicate records, invalid values, and inconsistent data types.
- Prepare a clean dataset while recording the treatment applied to incomplete observations.
- Calculate descriptive statistics such as mean, median, minimum, maximum, and standard deviation.
- Examine pollutant patterns across time and, where available, across stations or locations.
- Create readable charts that communicate trends and relationships without overstating causation.
- Export cleaned data and visualisations so the analysis can be reviewed or shared.
- Document limitations and recommend improvements for future data collection and analysis.

Expected deliverables

The expected deliverables are a documented Python script or notebook, a cleaned CSV file, descriptive-statistics output, a set of charts, and a report explaining the methodology, findings, limitations, and possible next steps.

4. Overview of Air Quality Data Analysis Concept Used

4.1 Air-quality measurements

Air-quality monitoring systems measure or estimate the concentration of pollutants in ambient air. PM2.5 refers to particulate matter with an aerodynamic diameter of 2.5 micrometres or less; PM10 refers to particles of 10 micrometres or less. Gaseous pollutants can include NO_2, SO_2, CO, and O_3. Each variable has its own unit and interpretation, so values should not be compared directly across pollutants unless the units and meaning make that comparison valid.

4.2 Data analytics concepts

Descriptive analytics summarises what the recorded data show. Exploratory data analysis (EDA) uses tables and charts to detect distributions, trends, relationships, and possible anomalies. Time-series analysis preserves the order of observations and is useful when readings are recorded by date or time. Data cleaning aims to improve consistency while avoiding the removal of legitimate extreme pollution events.

4.3 Typical dataset fields

A dataset may include timestamp, station name, city, PM2.5, PM10, NO2, SO2, CO, O3, temperature, humidity, and wind speed. This script treats timestamp as required and analyses whichever pollutant columns are actually present. Column names should be checked against the real file before running the project.

4.4 Data quality principles

Missing values should be measured and reported before being handled. Duplicate rows may represent repeated imports, but repeated timestamps can also be legitimate when multiple stations are recorded. Physical plausibility checks should use documented sensor ranges or domain rules rather than arbitrary deletion. Outliers should be investigated; a very high reading may be a real event, an instrument fault, or a data-entry error.

4.5 Ethical interpretation

The report should clearly distinguish observed association from causation. For example, a rise in particulate matter during a particular month does not by itself prove that one activity caused the rise. Strong causal claims require additional evidence, contextual information, and an appropriate research design.

6. Working Model / Process Flow

The workflow begins with a CSV file collected from a monitoring station, open-data portal, or approved academic dataset. The input file should be stored in the project folder and its source, download date, units, and licensing conditions should be recorded.

- Collect the dataset and record its source, location, time coverage, and units.
- Load the CSV using Pandas and inspect the first rows, column names, shape, and data types.
- Standardise column names and convert timestamp and pollutant fields to appropriate data types.
- Check missing values, duplicate rows, impossible values, and inconsistent timestamps.
- Apply documented cleaning rules; retain a raw copy so the original data remain available.
- Calculate descriptive statistics and, where appropriate, aggregate readings by day or month.
- Create charts for pollutant distributions, trends over time, and pollutant-to-pollutant comparisons.
- Export cleaned data, summary tables, and figures for inclusion in the report.
- Interpret the results with reference to data limitations and avoid unsupported causal claims.

Process-flow representation

```
Raw CSV → Load and inspect → Clean and validate → EDA and statistics → Visualise → Interpret → Export report outputs
```

Implementation notes

The code below is designed to be adaptable. It expects a file named air_quality.csv with a date or timestamp field. It detects common pollutant column names after normalising headers. If the supplied dataset uses different field names, update the candidate lists in the script. The script creates an output folder and saves its outputs there.

7. Benefits and Limitations

Benefits

• Repeatability: the same script can be rerun when a new dataset is received.

• Efficiency: Pandas can process many rows more quickly than manual spreadsheet inspection.

• Transparency: cleaning operations and assumptions can be documented in code.

• Clear communication: charts can reveal trends and unusual observations that are difficult to spot in a large table.

• Extensibility: the workflow can later include weather data, station comparisons, forecasting, or dashboards.

• Low cost: the core libraries used in the project are open source.

Limitations

• The quality of the conclusions depends on the reliability, completeness, and representativeness of the input data.

• A single monitoring station may not represent air quality across an entire city or region.

• Missing measurements can bias results, especially if they occur more often during particular conditions.

• Sensor calibration, station relocation, and changes in measurement methods may affect comparability.

• Descriptive analysis identifies patterns but does not automatically establish causes or predict future values.

• A result from one time period should not be generalised to other seasons or locations without additional evidence.

These limitations should be reported rather than hidden. A responsible analysis is valuable not because it claims certainty, but because it makes the available evidence and its uncertainty understandable.

8. Technology Used

Software and libraries

• Python 3: core programming language.

• Pandas: loading, cleaning, transforming, grouping, and summarising tabular data.

• NumPy: numerical support and handling of missing numeric values.

• Matplotlib: creation and saving of charts.

• Seaborn: statistical visualisation built on Matplotlib.

• A Python IDE: interactive development, testing, and documentation.

Hardware requirements

A standard laptop or desktop computer with sufficient storage for the dataset is adequate for a typical student-sized CSV file. Very large datasets may require additional memory or processing time, but this workflow does not require specialised hardware.

Installation

```
python -m pip install pandas numpy matplotlib seaborn
```

9. Complete Python implementation

Save the following program as air_quality_analysis.py in the same directory as air_quality.csv. It is written as a reusable baseline: it validates the input, cleans common formatting problems, generates statistics and charts, and saves outputs. The actual column names in the provided dataset must be checked before use.

```
from pathlib import Path
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


INPUT_FILE = Path("air_quality.csv")
OUTPUT_DIR = Path("air_quality_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)


# Candidate names are matched after headers are normalised.
DATE_CANDIDATES = ["date", "datetime", "timestamp", "time", "sampling_date"]
POLLUTANT_CANDIDATES = [
    "pm25", "pm2_5", "pm10", "no2", "so2", "co", "o3",
    "pm2.5", "nitrogen_dioxide", "sulphur_dioxide", "ozone",
    "carbon_monoxide"
]



def normalise_name(name):
    """Convert a column header to a simple lower-case identifier."""
    name = str(name).strip().lower()
    name = name.replace("₂", "2").replace("₃", "3")
    name = name.replace(".", "_").replace("-", "_").replace(" ", "_")
    name = re.sub(r"[^a-z0-9_]+", "", name)
    name = re.sub(r"_+", "_", name).strip("_")
    return name



def find_date_column(columns):
    for candidate in DATE_CANDIDATES:
        if candidate in columns:
            return candidate
    # Also accept a column containing the word date or time.
    for column in columns:
        if "date" in column or "time" in column:
            return column
    return None



def main():
```

```
    if not INPUT_FILE.exists():
        raise FileNotFoundError(
            f"Cannot find {INPUT_FILE}. Put air_quality.csv beside this script."
        )

    df = pd.read_csv(INPUT_FILE)
    if df.empty:
        raise ValueError("The input CSV has no data rows.")

    # Keep a record of original headers, then standardise them.
    original_columns = list(df.columns)
    df.columns = [normalise_name(c) for c in df.columns]
    if len(set(df.columns)) != len(df.columns):
        raise ValueError("Column names collide after normalisation; rename them manually.")

    print("Original columns:", original_columns)
    print("Dataset shape:", df.shape)
    print("
First five rows:
", df.head())
    print("
Data types before cleaning:
", df.dtypes)

    date_col = find_date_column(df.columns)
    if date_col is None:
        raise ValueError("No date/time column found. Rename it to date or timestamp.")

    df[date_col] = pd.to_datetime(df[date_col], errors="coerce")
    invalid_dates = int(df[date_col].isna().sum())
    df = df.dropna(subset=[date_col]).copy()
    df = df.sort_values(date_col)

    # Identify pollutant columns that are actually present.
    pollutant_cols = [c for c in df.columns if c in POLLUTANT_CANDIDATES]
    # Add likely pollutant columns using name fragments, avoiding station/date fields.
    fragments = ("pm25", "pm2_5", "pm10", "no2", "so2", "co", "o3",
                 "nitrogen_dioxide", "sulphur_dioxide", "carbon_monoxide", "ozone")
    for c in df.columns:
        if c != date_col and any(fragment in c for fragment in fragments):
            if c not in pollutant_cols:
                pollutant_cols.append(c)

    if not pollutant_cols:
        raise ValueError(
            "No pollutant columns detected. Update POLLUTANT_CANDIDATES "
            "to match the CSV headers."
        )

    # Convert pollutant measurements to numbers; unparseable text becomes NaN.
    for column in pollutant_cols:
        df[column] = pd.to_numeric(df[column], errors="coerce")
        # Negative concentrations are generally invalid for these measurements.
        # Treat them as missing for analysis, but retain the count for reporting.
        negative_count = int((df[column] < 0).sum())
        if negative_count:
            print(f"{column}: {negative_count} negative values set to missing")
            df.loc[df[column] < 0, column] = np.nan

    duplicate_count = int(df.duplicated().sum())
    df = df.drop_duplicates().copy()

    quality = pd.DataFrame({
        "missing_count": df.isna().sum(),
        "missing_percent": (df.isna().mean() * 100).round(2),
        "dtype": df.dtypes.astype(str)
    })
```

```
    quality.to_csv(OUTPUT_DIR / "data_quality_summary.csv")

    # Descriptive statistics for the pollutant fields present in this file.
    stats = df[pollutant_cols].describe().T
    stats["median"] = df[pollutant_cols].median()
    stats["missing_count"] = df[pollutant_cols].isna().sum()
    stats.to_csv(OUTPUT_DIR / "pollutant_descriptive_statistics.csv")

    # Save the cleaned data; missing pollutant readings remain missing rather
    # than being silently replaced by zero or a fabricated value.
    df.to_csv(OUTPUT_DIR / "cleaned_air_quality.csv", index=False)

    print("
Rows with invalid dates removed:", invalid_dates)
    print("Duplicate rows removed:", duplicate_count)
    print("Date range:", df[date_col].min(), "to", df[date_col].max())
    print("Pollutant columns:", pollutant_cols)
    print("
Descriptive statistics:
", stats)

    sns.set_theme()
    # Chart 1: time trends. Daily mean helps reduce clutter from dense readings.
    for column in pollutant_cols:
        series = df[[date_col, column]].dropna().set_index(date_col)[column]
        if series.empty:
            print(f"Skipping {column}: no valid readings")
            continue
        daily = series.resample("D").mean()
        plt.figure(figsize=(11, 5))
        plt.plot(daily.index, daily.values, linewidth=1.4)
        plt.title(f"Daily Mean of {column.upper()} Over Time")
        plt.xlabel("Date")
        plt.ylabel(f"{column.upper()} concentration (dataset units)")
        plt.tight_layout()
        plt.savefig(OUTPUT_DIR / f"{column}_daily_trend.png", dpi=160)
        plt.close()

        # Chart 2: distribution of observed readings.
        plt.figure(figsize=(8, 5))
        sns.histplot(df[column].dropna(), bins=30, kde=True)
        plt.title(f"Distribution of {column.upper()} Readings")
        plt.xlabel(f"{column.upper()} concentration (dataset units)")
        plt.ylabel("Frequency")
        plt.tight_layout()
        plt.savefig(OUTPUT_DIR / f"{column}_distribution.png", dpi=160)
        plt.close()

    # Correlation is calculated only where at least two pollutant columns exist.
    if len(pollutant_cols) >= 2:
        corr = df[pollutant_cols].corr(numeric_only=True)
        corr.to_csv(OUTPUT_DIR / "pollutant_correlation_matrix.csv")
        plt.figure(figsize=(9, 7))
        sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
        plt.title("Correlation Between Pollutant Measurements")
        plt.tight_layout()
        plt.savefig(OUTPUT_DIR / "pollutant_correlation_heatmap.png", dpi=160)
        plt.close()

    print(f"
Analysis complete. Results saved in: {OUTPUT_DIR.resolve()}")


if __name__ == "__main__":
    main()
```

How to run the program

- Install Python 3 and the libraries listed above.
- Place the CSV file in the same folder as the script and name it air_quality.csv.
- Open a terminal in that folder and run: python air_quality_analysis.py
- Review the printed summary and inspect the air_quality_outputs folder.
- If the program cannot detect the date or pollutant columns, update the candidate lists to match the actual headers.

Example of a suitable input CSV

```
timestamp,station,PM2.5,PM10,NO2,O3
2025-01-01 08:00,Station_A,42.1,67.0,18.5,21.4
2025-01-01 09:00,Station_A,45.3,70.2,19.1,22.0
2025-01-01 10:00,Station_A,39.8,63.4,17.7,23.6
```

The values above are illustrative examples for demonstrating the file structure only. They are not official observations and must not be used as evidence about a real location. Replace them with a dataset whose source and units can be verified.

10. Risk and Security Measures

Data integrity risks

Incorrect timestamps, mixed units, duplicate imports, missing values, and faulty sensors can affect the analysis. The program reports data-quality information and keeps missing values as missing. The original raw file should be preserved, and any cleaning decisions should be recorded so that the results can be reproduced.

Privacy and access

Most ambient air-quality readings are environmental measurements, but some datasets may contain precise station locations, operational details, or information linked to individuals. Only data necessary for the project should be collected. Private credentials, API keys, and personal information should not be committed to a public repository.

File and code safety

- Use trusted datasets and inspect file sources before opening or processing them.
- Keep an untouched copy of the raw data and write generated files to a separate output folder.
- Do not execute unknown code copied from an unverified source.
- Use version control to track code changes; avoid uploading confidential or restricted data.
- Document package versions and the date of analysis where reproducibility matters.

Analytical risks

A misleading chart can result from inconsistent units, an inappropriate aggregation method, or an unreported gap in data. Correlation between two pollutants does not prove that one pollutant causes the other. Thresholds for categorising readings should come from an identified standard appropriate to the jurisdiction and averaging period, not from arbitrary cut-offs. Before publishing conclusions, verify the dataset metadata and review results for plausibility.

11. Findings / Analysis

The findings section must be completed using the actual output from the selected dataset. Because the input CSV was not included with the report template, this case study does not invent measured averages, dates of peak pollution, or city-specific conclusions. The script produces the numerical outputs needed to write evidence-based findings.

How to interpret the generated statistics

• Mean: the arithmetic average across valid readings; it can be affected by extreme values.

• Median: the middle observation; it is often useful when the distribution is skewed.

• Standard deviation: indicates how widely readings vary around their mean.

• Minimum and maximum: identify the range, but extreme values should be checked against source records before being labelled errors.

• Missing percentage: shows how much information is unavailable for each field and helps assess whether comparisons are fair.

Trend-chart analysis

A daily-mean line chart can show whether a pollutant rises, falls, or varies across the available period. Gaps in the line may reflect days without valid measurements rather than clean air. Compare similar time periods when evaluating locations, and account for differences in monitoring coverage. If the dataset covers only a few days, it cannot support reliable seasonal conclusions.

Distribution analysis

A histogram helps reveal whether readings cluster in a narrow range, have a long right tail, or include unusual extremes. A right-skewed distribution is common in environmental measurements, but the shape must be confirmed from the actual chart. Do not remove high readings merely because they are inconvenient; first check for sensor faults, unit mistakes, and supporting contextual evidence.

Correlation analysis

A correlation matrix summarises linear relationships between pollutant measurements that occur in the same rows. Positive correlation means two measurements tend to rise together in the sample; negative correlation means one tends to be lower when the other is higher. Correlations may reflect shared sources, weather, time-of-day patterns, or sampling design. They do not establish causation, and pairwise missing data can change the sample used for each coefficient.

Results table to complete after running the code

Enter the values from pollutant_descriptive_statistics.csv and data_quality_summary.csv. Include the dataset name, source, measurement units, date range, and number of valid readings in the final submitted version.

Pollutant	Mean	Median	Minimum	Maximum	Missing count
PM2.5					
PM10					
NO2					
SO2					
CO					
O3					

Discussion template

After running the program, discuss (1) which pollutants are available and their units, (2) which variables have the most missing data, (3) how pollutant concentrations change over time, (4) whether the distributions contain unusually high observations, and (5) what additional information is required to explain the patterns. Only discuss pollutants that are present in the dataset, and support each claim with a statistic, chart, or source record.

Practical significance

A repeatable analysis pipeline can support environmental awareness, student research, preliminary screening, and communication with decision-makers. However, this project is an educational analysis and not a substitute for official monitoring, medical advice, or regulatory assessment. Any public-facing claims should be checked against authoritative monitoring data and applicable standards.

12. Conclusion

This case study presents a structured Python workflow for analysing air-quality data. It demonstrates how to load a dataset, standardise column names, convert dates and measurements, examine data quality, calculate descriptive statistics, create time-series and distribution charts, explore correlations, and export outputs. The approach is repeatable and can be adapted to different monitoring datasets by updating the column mappings.

The reliability of the final conclusions depends on the data source, coverage, measurement units, sensor quality, and cleaning decisions. The program deliberately avoids replacing missing measurements with zero and does not automatically delete extreme readings simply because they appear unusual. These choices help reduce the risk of creating false patterns.

The next stage is to run the script on a verified dataset, review the generated charts and tables, and replace the blank results table with measured values. Further work could include joining weather observations, comparing stations, mapping spatial differences, evaluating air-quality standards, or testing forecasting methods on a sufficiently long and reliable time series.

In conclusion, Python provides a practical foundation for environmental data analysis, but responsible interpretation remains essential. Good analysis combines code with transparent methods, source verification, appropriate visualisation, and honest discussion of uncertainty.

13. Learning Outcomes

- Learned how to structure an end-to-end data-analysis task using Python.
- Practised importing CSV data and inspecting DataFrame structure and data types.
- Understood why date conversion, numeric conversion, duplicate checks, and missing-value assessment are necessary.
- Used descriptive statistics to summarise distributions and variability.
- Learned to produce line charts, histograms, and correlation heatmaps.
- Recognised that correlation does not establish causation and that outliers require investigation.
- Practised exporting analysis results so they can be reviewed and reproduced.
- Understood the importance of recording sources, units, limitations, and cleaning decisions.

14. References

1. Pandas documentation, “Getting started” and user guide. https://pandas.pydata.org/docs/

2. NumPy documentation. https://numpy.org/doc/

3. Matplotlib documentation. https://matplotlib.org/stable/

4. Seaborn documentation. https://seaborn.pydata.org/

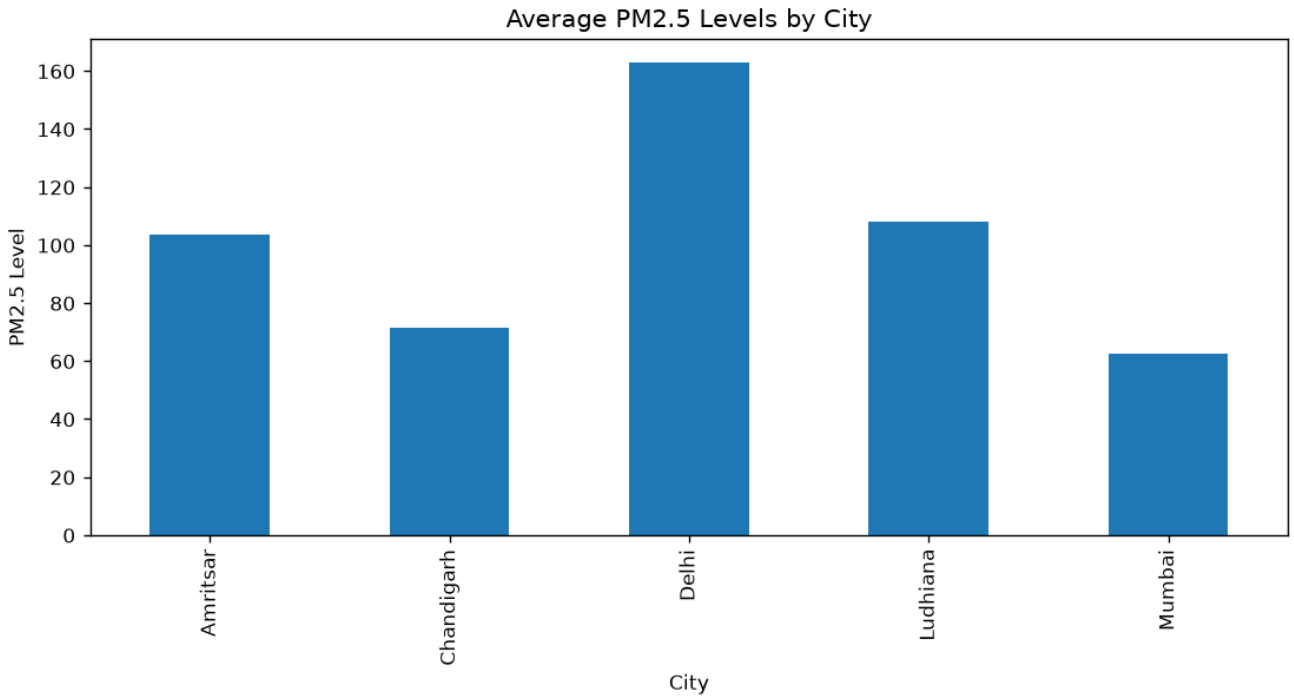
5. World Health Organization, “WHO global air quality guidelines” (2021). https://www.who.int/publications/i/item/9789240034228

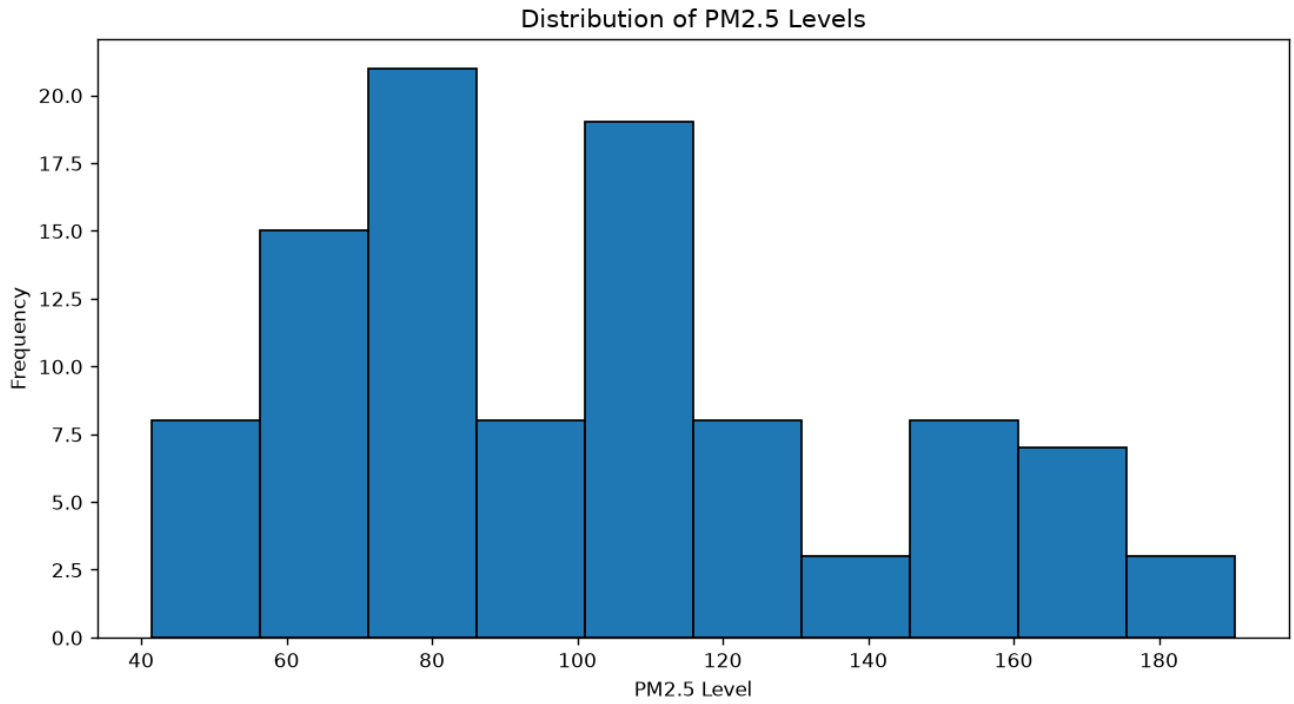
6. United States Environmental Protection Agency, “Air Quality System” and air monitoring resources. https://www.epa.gov/aqs

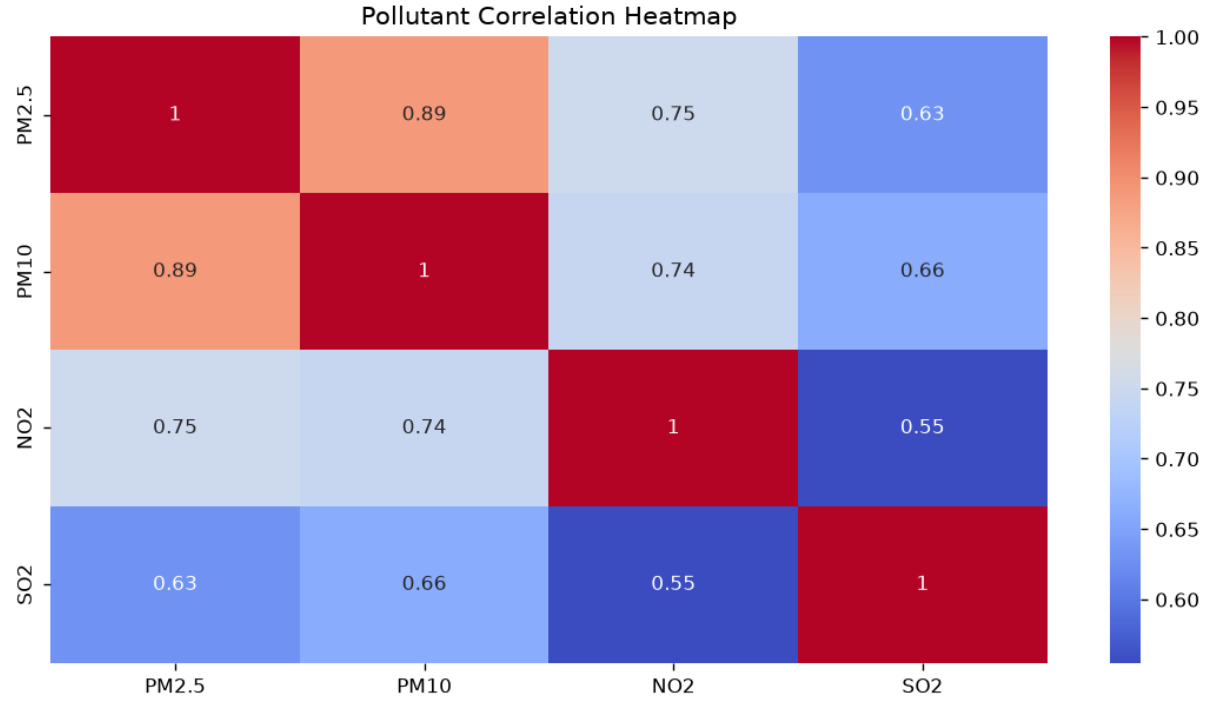
7. Government or official open-data portal used for the selected dataset: add the exact dataset title, publisher, URL, download date, licence, and station metadata here.

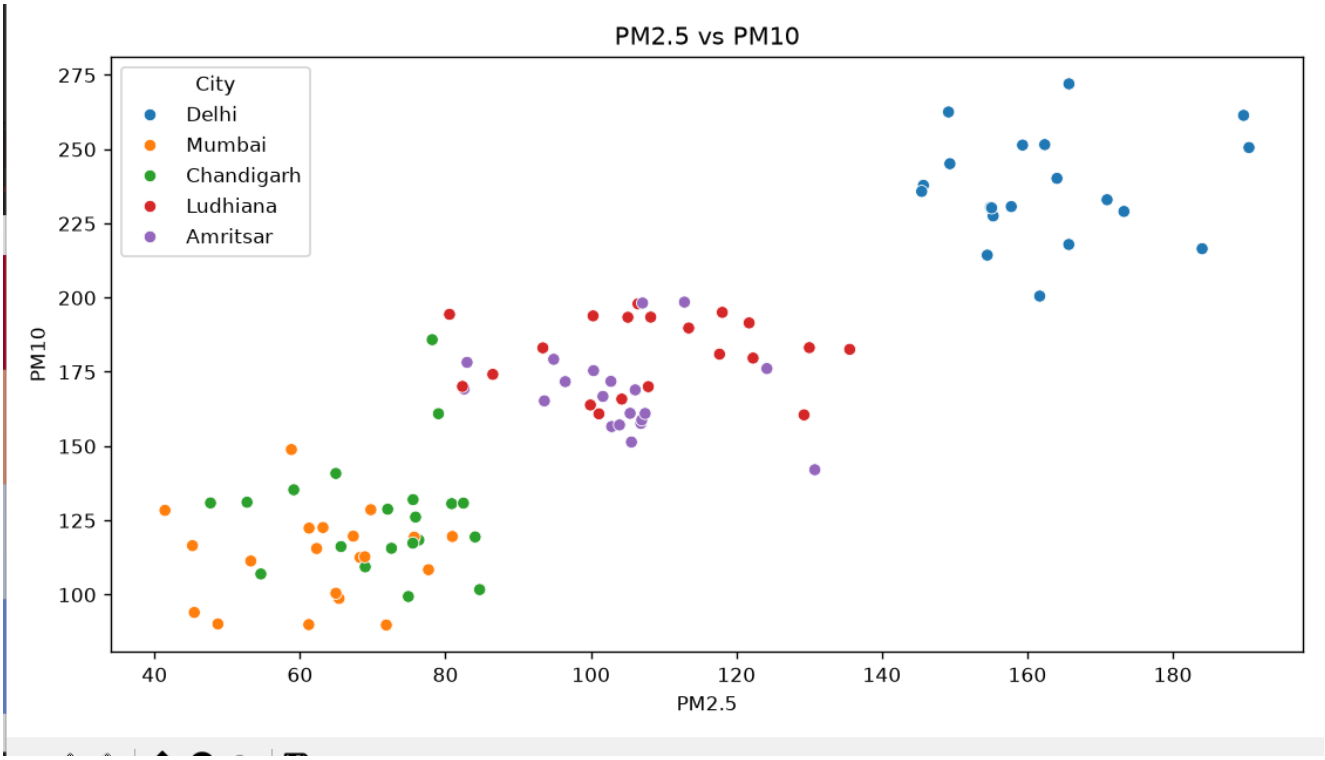
Access dates and dataset details should be completed at the time the sources are consulted. Follow the terms of use for the selected data source.

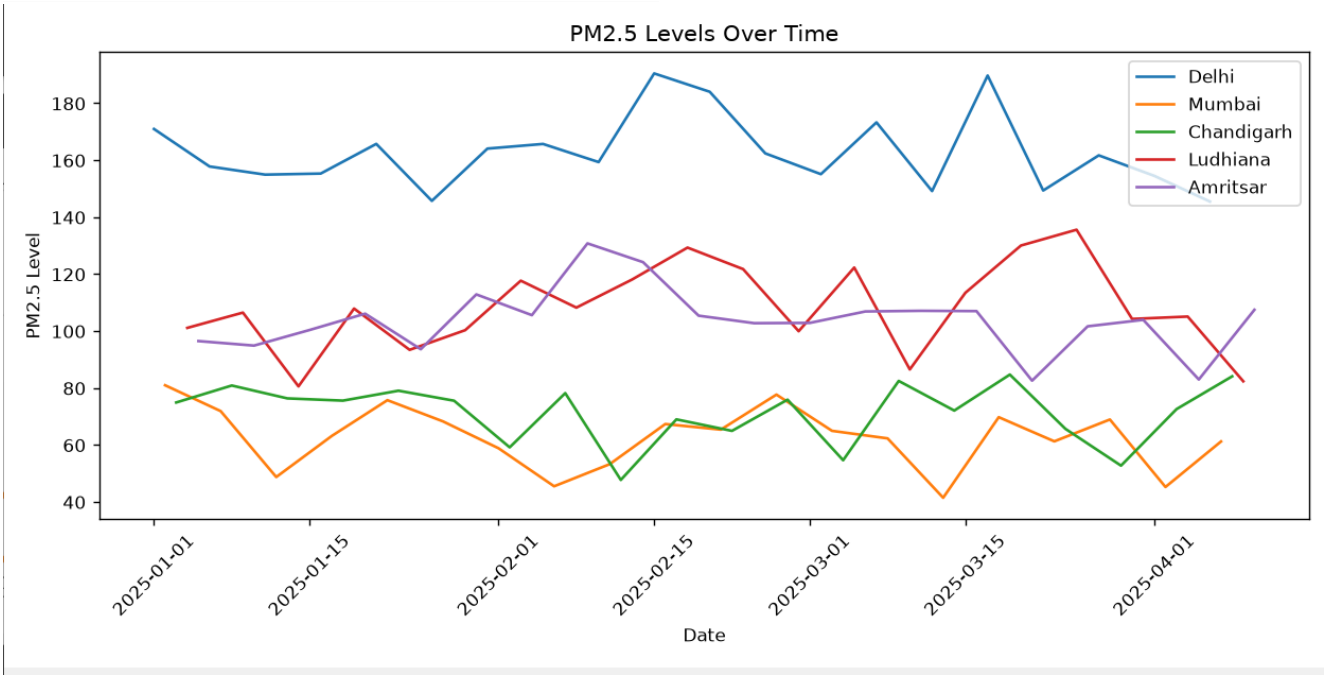
15. Screenshots

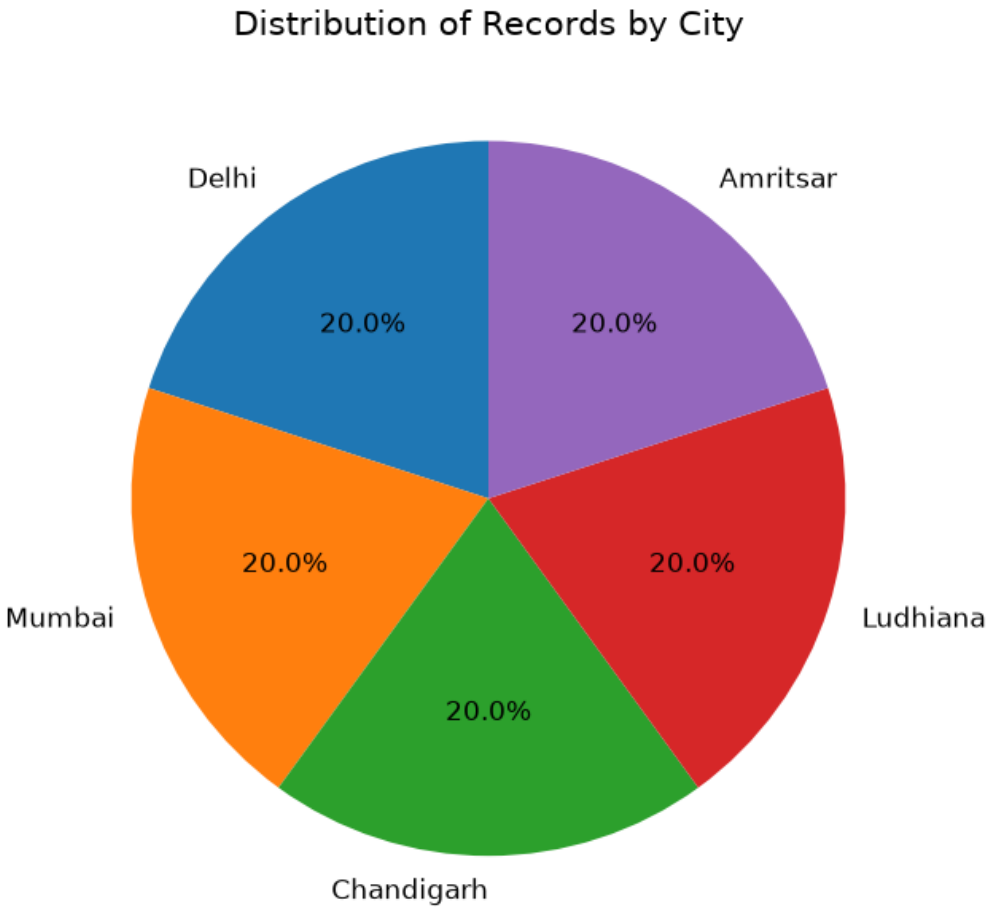

Average PM2.5 Levels by City
PM2.5 Level
0
20
40
60
80
100
120
140
160
Amritsar
Chandigarh
Delhi
Ludhiana
Mumbai
City

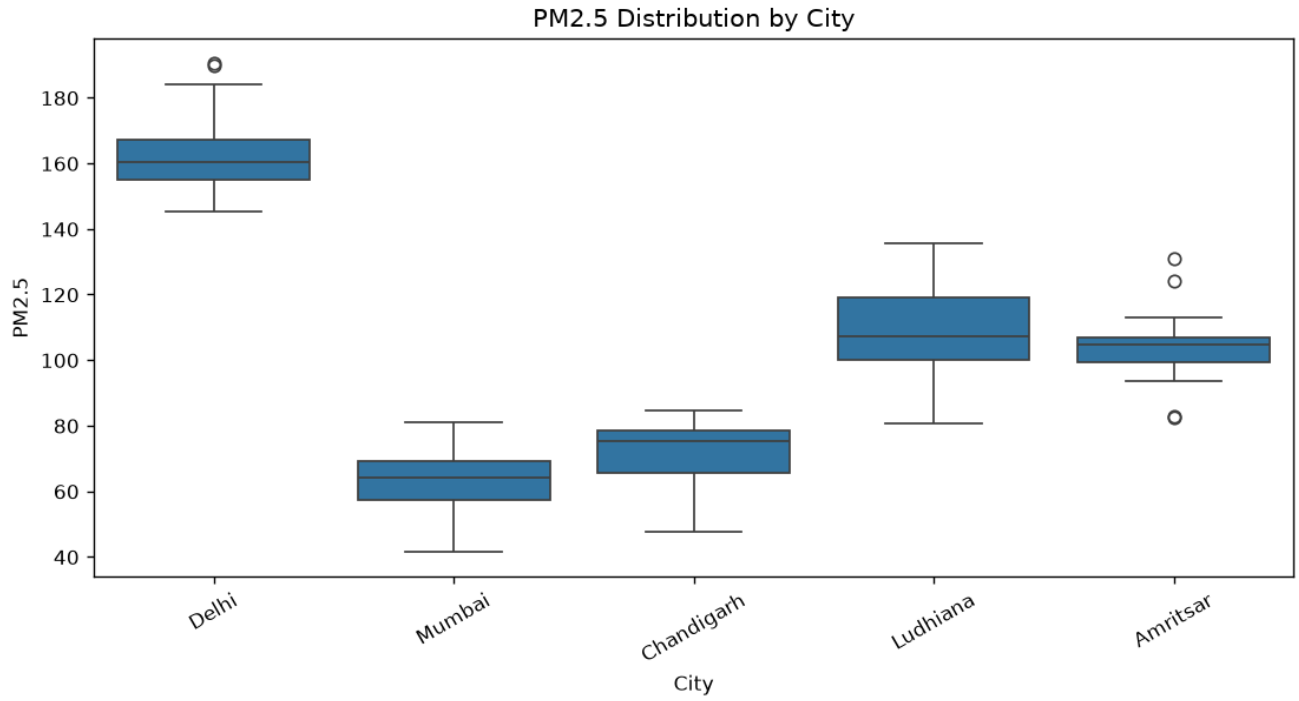

Distribution of PM2.5 Levels
Frequency
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
40
60
80
100
120
140
160
180
PM2.5 Level

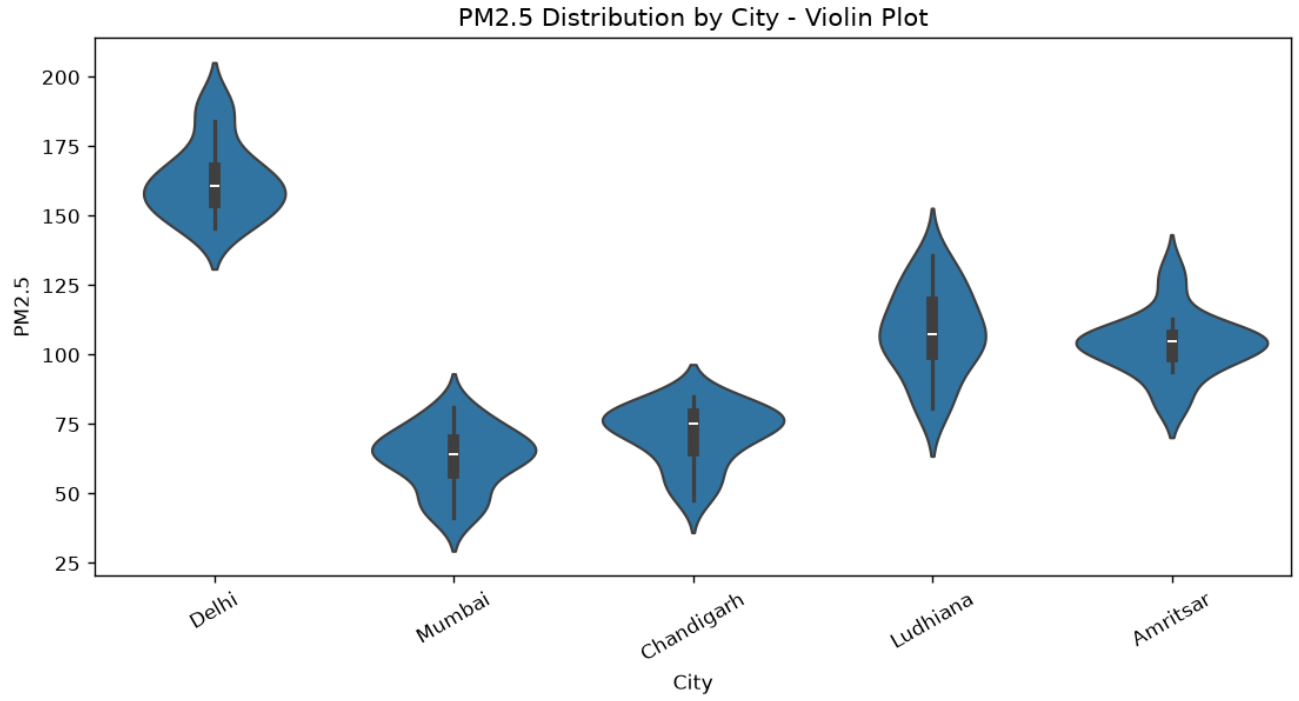

Pollutant Correlation Heatmap
PM2.5
PM10
NO2
SO2
1
0.89
0.75
0.63
0.89
1
0.74
0.66
0.75
0.74
1
0.55
0.63
0.66
0.55
1
PM2.5
PM10
NO2
SO2
1.00
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.60

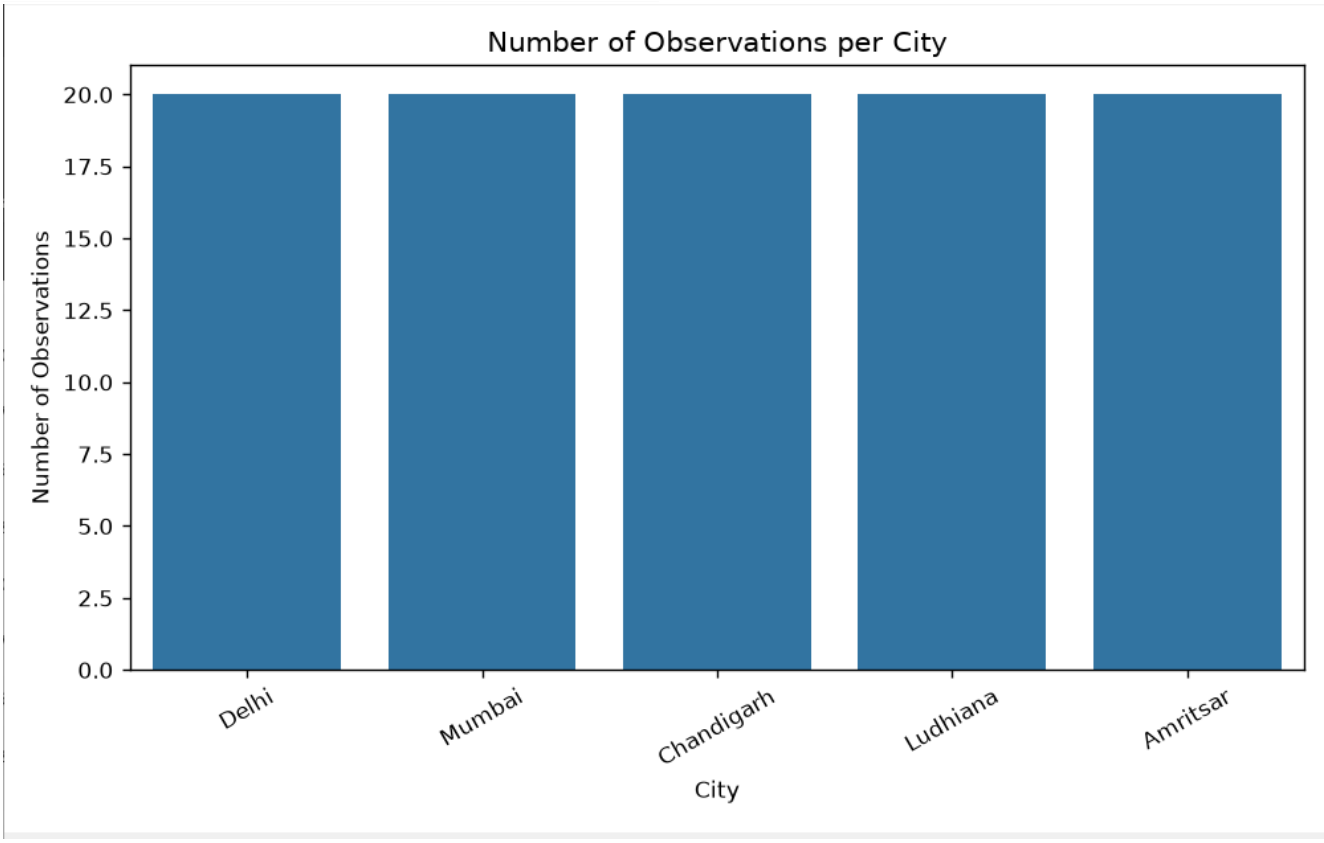

PM2.5 vs PM10
City
Delhi
Mumbai
Chandigarh
Ludhiana
Amritsar
PM10
PM2.5
275
250
225
200
175
150
125
100
40
60
80
100
120
140
160
180

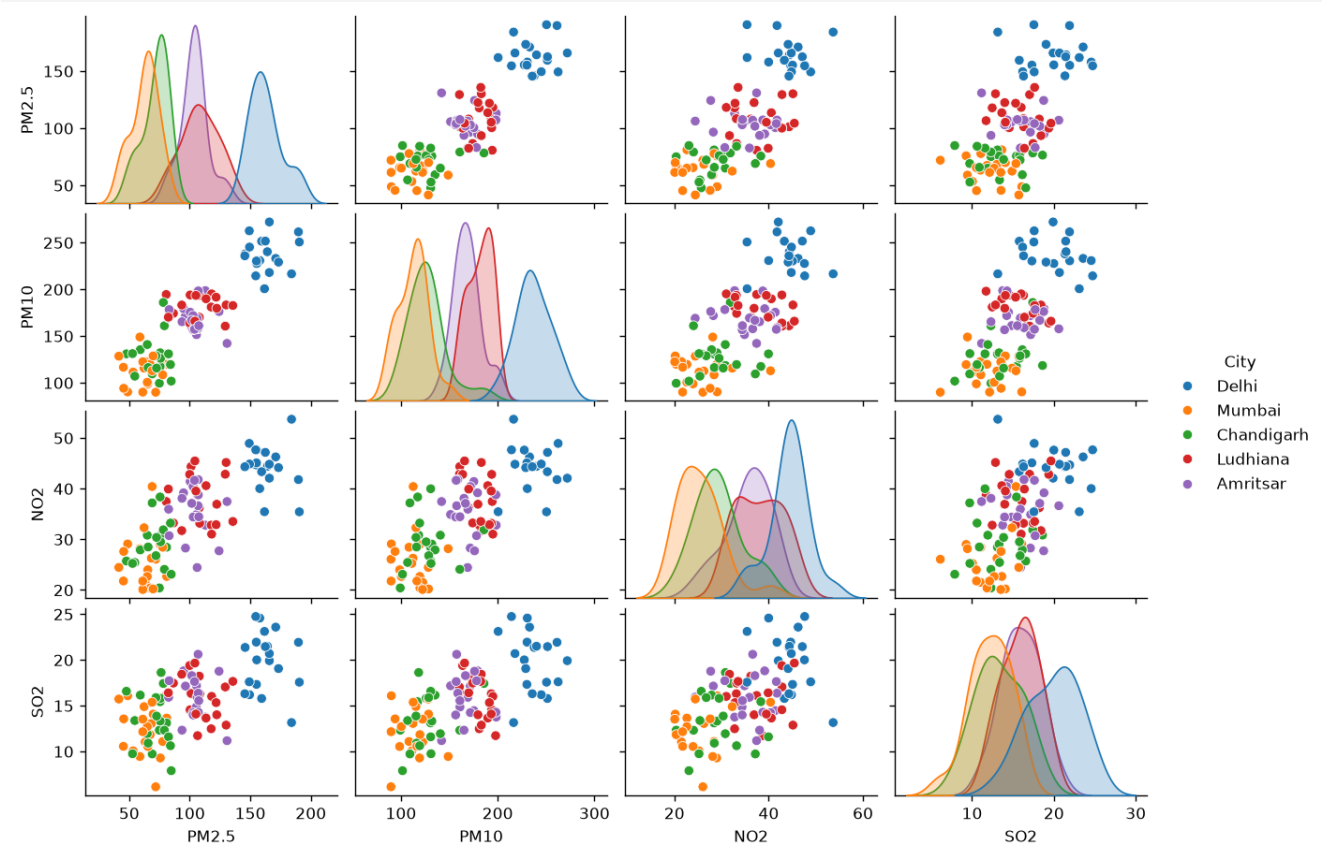

PM2.5 Levels Over Time
PM2.5 Level
Date
Delhi
Mumbai
Chandigarh
Ludhiana
Amritsar
180
160
140
120
100
80
60
40
2025-01-01
2025-01-15
2025-02-01
2025-02-15
2025-03-01
2025-03-15
2025-04-01


Distribution of Records by City
Delhi
Amritsar
Mumbai
Ludhiana
Chandigarh
20.0%
20.0%
20.0%
20.0%
20.0%


PM2.5 Distribution by City
PM2.5
180
160
140
120
100
80
60
40
Delhi
Mumbai
Chandigarh
Ludhiana
Amritsar
City


PM2.5 Distribution by City - Violin Plot
PM2.5
200
175
150
125
100
75
50
25
Delhi
Mumbai
Chandigarh
Ludhiana
Amritsar
City





12. Evaluation Grid (To be created as per the SOP and Assessment guidelines by the faculty):

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Suggested project folder structure

```
air-quality-project/
  air_quality_analysis.py
  air_quality.csv                 # input dataset; check sharing licence
  air_quality_outputs/
    cleaned_air_quality.csv
    data_quality_summary.csv
    pollutant_descriptive_statistics.csv
    pollutant_daily_trend.png
    pollutant_distribution.png
    pollutant_correlation_matrix.csv
    pollutant_correlation_heatmap.png
  README.md
```

The evaluation grid below is retained from the supplied template. Marks and signatures are for the faculty to complete.

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	PROJECT TITLE		2 Marks
2.	CASE STUDY		5 Marks
3.	Github Upload Link		1 Marks
4.	Blog Upload Link		1 Marks
5.	Follow Format		1 Marks
	TOTAL		10 Marks
	AVG		6 Marks

Teacher Signature